Enterprise Software Engineer Technical Assessment

Position: Software Engineer (.NET)

Introduction

Thank you for participating in our technical assessment.

The goal of this exercise is to evaluate your ability to design and build an enterprise-grade software solution. We value clean architecture, maintainable code, scalability, security, testing, and sound engineering decisions as much as functional completeness.

Where requirements are intentionally open-ended, make reasonable assumptions and document them in your `README.md`.

Objective

Develop a **Library Management System** consisting of:

- RESTful Backend API
- Responsive Web Application

Functional Requirements

Implement the following modules:

- Authentication (JWT) & Role-based Authorization
- Branch Management
- Book Management
- Member Management
- Borrow & Return Management
- Reservation Queue
- Reports

The application should support standard CRUD operations, search/filtering where appropriate, and basic business validations.

Frontend Requirements

Your application should include:

- Login / Logout
- Dashboard
- Role-based Navigation
- Branch, Book & Member Management
- Borrow & Return
- Reservation Queue
- Reports
- Responsive UI

Technical Expectations

Your solution should demonstrate good software engineering practices, including:

- Clean/Onion Architecture

- Dependency Injection
- SOLID Principles
- Appropriate design patterns (e.g., Repository, Specification, Strategy, Factory, MediatR/CQRS)
- FluentValidation
- Centralized Exception Handling
- Logging (Serilog or equivalent)
- Secure Coding Practices
- Asynchronous Programming
- Efficient Database Access
- Unit Testing

Recommended Stack

- ASP.NET Core (.NET 8+)
- Entity Framework Core
- PostgreSQL
- Swagger/OpenAPI
- React / Angular / Vue / Blazor
- xUnit / NUnit

Deliverables

Your submission should include:

- Source Code
- Git Repository
- README.md
- Database Migrations
- Swagger/OpenAPI (or Postman Collection)
- Unit Tests
- Setup Instructions

Evaluation Criteria

Category	Marks
Functional Requirements	25
Frontend Implementation	10
Architecture & Project Structure	15
Code Quality & Maintainability	10
SOLID & Dependency Injection	10
Design Patterns	10
Database Design	5
Security	5
Performance	5
Unit Testing	5

Documentation & Git Practices	10
Total	100

Bonus (Optional)

Additional credit will be given for implementing features such as:

- CQRS
- Domain Events
- Optimistic Concurrency
- API Versioning
- Health Checks
- Docker
- Redis
- Background Jobs
- Excel/PDF Export
- Email Notifications
- CI/CD Pipeline

Submission Guidelines

1. Create a **GitHub** or **GitLab** repository for your solution.
2. Ensure the repository is **public**, or grant access to the reviewers if it is private.
3. Include a comprehensive `README.md` with:
    - Setup instructions
    - Assumptions and design decisions
    - Environment configuration
    - How to run the application and tests
4. Do **not** commit secrets, passwords, API keys, or production configuration.
5. Email your submission to **[your-email@example.com]** with:
    - Full Name
    - Position Applied For
    - GitHub/GitLab Repository URL
    - A brief summary of your implementation and any bonus features

General Notes

- Focus on **quality over quantity**.
- Write clean, maintainable, and well-documented code.
- We value thoughtful architecture and engineering decisions over implementing every possible feature.
- AI-assisted development tools are permitted, but you should be able to explain and justify your implementation during the technical interview.